

연구 파이프라인 상세 기술 문서

프로젝트: 에틸렌 유도 이소플라보노이드 생합성 연구

작성일: 2026년 1월 15일

목적: 동료 연구자와의 기술 논의용

목차

- [연구 개요](#)
- [데이터 소스 및 전처리](#)
- [그래프 구축](#)
- [GNN 모델 아키텍처](#)
- [모델 학습 파이프라인](#)
- [해석 가능성 분석 \(GNNShap\)](#)
- [분자 도킹 파이프라인](#)
- [MD 시뮬레이션 계획](#)
- [검증 및 평가 지표](#)
- [파일 구조 및 재현성](#)

1. 연구 개요

1.1 연구 배경

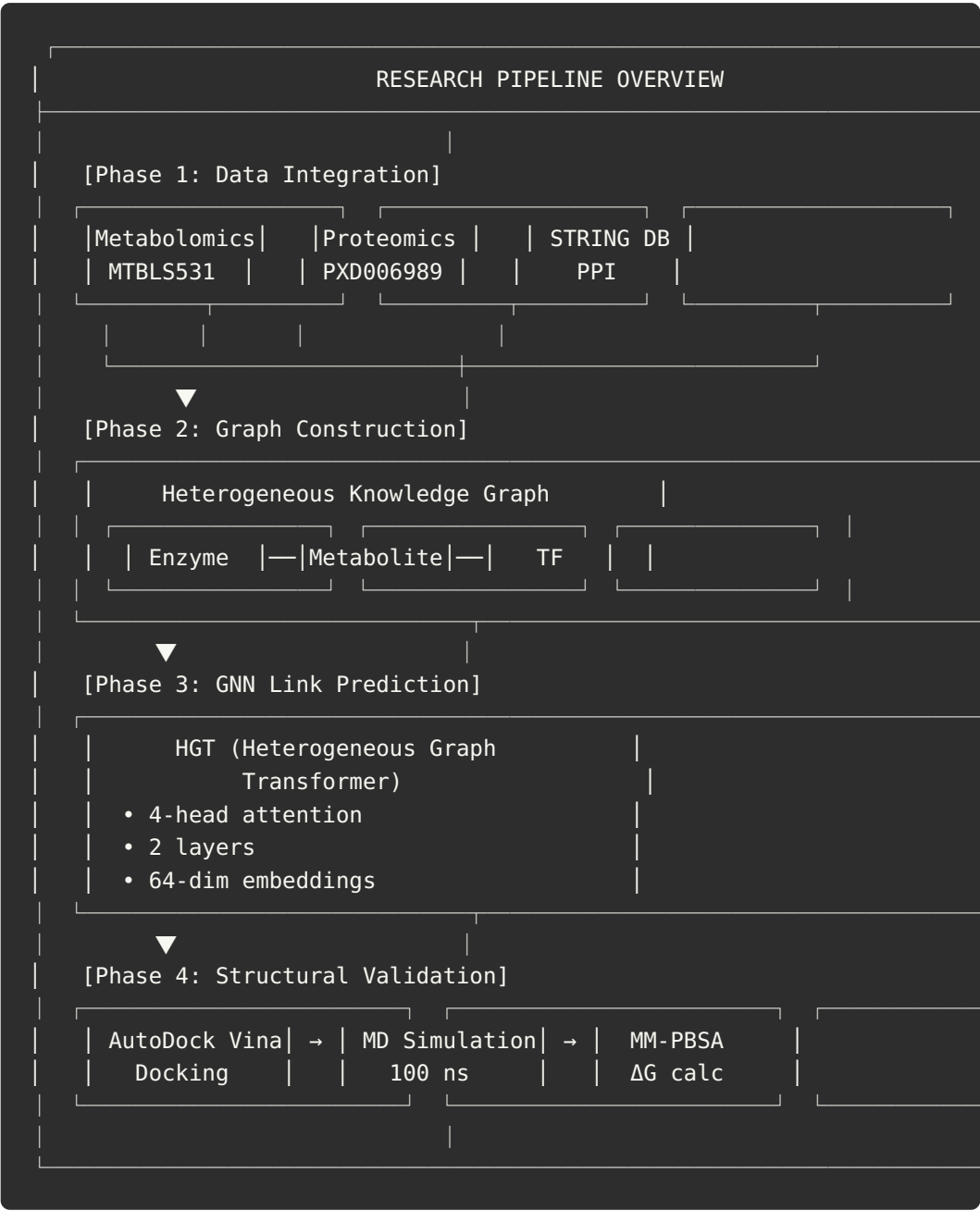
에틸렌 처리된 콩(Glycine max) 잎에서 말로닐화/아세틸화 이소플라본 접합체가 4,000배 이상 증가하는 현상을 발견하였습니다. 본 연구는 이 현상의 분자적 메커니즘을 규명하기 위해 멀티오믹스 데이터와 기계학습을 통합합니다.

1.2 연구 목표

- 대사체-효소 상호작용 예측: GNN 기반 link prediction
- 구조적 검증: 분자 도킹을 통한 결합 가능성 평가
- 동적 안정성 검증: MD 시뮬레이션을 통한 결합 안정성 확인

4. 신규 타겟 발굴: 문헌에 보고되지 않은 상호작용 발견

1.3 전체 파이프라인 개요



2. 데이터 소스 및 전처리

2.1 멀티오믹스 데이터

대사체학 (Metabolomics)

항목	내용
데이터셋	MTBLS531 (MetaboLights)
분석 장비	LC-MS/GC-MS
처리 파일	data/processed/mtbls531_differential.csv
대사체 수	79개
유의 대사체	43개 (54%, $P < 0.05$)

주요 발견: - 최대 Fold Change: $12\text{-}13 \times (\text{Log}2) = 4,000\text{-}8,000 \times (\text{linear})$ - Top 대사체: 6''-O-Malonyldaidzin, 6''-O-Malonylgenistin, 6''-O-Acetyldaidzin

단백체학 (Proteomics)

항목	내용
데이터셋	PXD006989 (PRIDE)
분석 장비	nLC-MS/MS
처리 파일	data/processed/pxd006989_differential.csv
단백질 수	>6,000개
핵심 효소	6개 (이소플라보노이드 경로)

핵심 효소 발현 변화:

효소	약자	Fold Change	P-value	기능
Isoflavone Reductase	IFR	$6.4 \times$	<0.01	이소플라본 환원
Chalcone Isomerase	CHI	$5.1 \times$	<0.01	칼콘→플라바논 이성화
4-Coumarate CoA Ligase	4CL	$3.9 \times$	<0.05	CoA 에스터 형성
Phenylalanine Ammonia Lyase	PAL	$3.7 \times$	<0.05	페닐알라닌 탈아민화

효소	약자	Fold Change	P-value	기능
Isoflavone Synthase	IFS	3.2×	<0.05	이소플라본 합성
Chalcone Synthase	CHS	2.9×	<0.05	칼콘 합성

2.2 단백질 상호작용 데이터

STRING Database (PPI)

항목	내용
종	Glycine max (Soybean)
Confidence Threshold	0.7 (high confidence)
처리 스크립트	src/dataloader.py (StringDBLoader)

```
# StringDBLoader 주요 기능
class StringDBLoader:
    def __init__(self, species_id=3847, score_threshold=700):
        """
        species_id: 3847 = Glycine max
        score_threshold: 700 = high confidence (0.7)
        """
```

2.3 구조 데이터

수용체 (PDB 구조)

PDB ID	단백질	해상도	유기체	경로
6YN7	β-Glucosidase	1.8 Å	Alicyclobacillus herbarius	data/structures/pdb/6YN7.pdb
8E83	2-HIS (IFS homolog)	2.0 Å	Medicago truncatula	data/structures/pdb/8E83.pdb
8EA1	2-HID	2.4 Å	Pueraria lobata	data/structures/pdb/8EA1.pdb
1EYQ	CHI	1.85 Å	Medicago sativa	data/structures/pdb/1EYQ.pdb

리간드 (SDF 구조)

화합물	PubChem CID	경로
6''-O-Malonyldaidzin	5318574	data/structures/ligands/6-0-Malonyldaidzin_CID5318574.sdf
6''-O-Malonylgenistin	5318568	data/structures/ligands/6-0-Malonylgenistin_CID5318568.sdf
6''-O-Acetyldaidzin	14034712	data/structures/ligands/6-0-Acetyldaidzin_CID14034712.sdf
6''-O-Acetylgenistin	5320413	data/structures/ligands/6-0-Acetylgenistin_CID5320413.sdf

3. 그래프 구축

3.1 이종 그래프 (Heterogeneous Graph) 설계

우리 그래프는 다양한 노드 타입과 엣지 타입을 포함하는 **이종 지식 그래프**입니다.

Graph Schema:				
Node Types:				
Type	Description	Feature Dim	Count	
Enzyme	생합성 효소	64	~500	
Metabolite	대사체 (이소플라본 등)	64	~200	
TF	전사인자 (ERF 등)	64	~100	
Edge Types:				
Edge Type	Description			
(Enzyme, catalyzes, Metabolite)	효소-대사체 촉매 관계			
(TF, regulates, Enzyme)	전사인자-효소 조절 관계			
(Enzyme, interacts, Enzyme)	효소-효소 상호작용 (PPI)			
(Metabolite, precursor, Met)	대사체 전구체 관계			

3.2 그래프 구축 스크립트

파일 위치: `src/bipartite_builder.py`

```
def build_bipartite_graph(graph_path, output_path):
    """
    Heterogeneous Graph (PPI + Metabolite-Enzyme) 구축

    Process:
    1. STRING PPI 그래프 로드
    2. 대사체 노드 추가 (200개)
    3. Enzyme-Metabolite 엣지 생성
    4. MSI Level 기반 엣지 가중치 할당
    5. 양방향 엣지 추가 (message passing용)
    """

    # MSI Level → Weight 매핑
    # Level 2 (Strong evidence): 1.0
    # Level 3 (Moderate): 0.7
    # Level 4 (Weak): 0.4
    def get_msi_weight(met_idx):
        pathway = met_pathways.get(compounds[met_idx], 'Other')
        if pathway in ['Phenylpropanoid', 'Flavonoid']:
            return 1.0 # High confidence
        else:
            return 0.4 # Lower confidence
```

3.3 특징 벡터 초기화

노드 타입	초기화 방법	차원
Enzyme	Random Normal	64
Metabolite	Random Normal (향후 분자기술자 사용 가능)	64
TF	Random Normal	64

향후 개선점: 분자 fingerprint (Morgan/ECFP), ESM-2 protein embeddings 사용 가능

4. GNN 모델 아키텍처

4.1 구현된 모델 옵션

파일 위치: `src/model.py`

모델	클래스명	설명	사용 사례
HGT 	HGT	Heterogeneous Graph Transformer	메인 모델
HAN	HAN	Heterogeneous Attention Network	비교 실험
HeteroSAGE	HeteroSAGE	Heterogeneous GraphSAGE	Baseline
SimpleMLP	SimpleMLP	Multi-Layer Perceptron (그래프 무시)	Ablation

4.2 HGT (메인 모델) 상세 설명

```
class HGT(nn.Module):
    """
        Heterogeneous Graph Transformer

        논문: "Heterogeneous Graph Transformer" (WWW 2020)

        핵심 특징:
        - 노드/엣지 타입별 서로 다른 attention 파라미터
          - Multi-head attention mechanism
          - Type-aware message passing
        """

    def __init__(self, metadata, in_channels, hidden_channels,
                 out_channels, num_heads=4, num_layers=2):
        super().__init__()

        # 1. 노드 타입별 입력 투영 레이어
        self.lin_dict = nn.ModuleDict()
        for node_type in metadata[0]:
            self.lin_dict[node_type] = Linear(in_channels, hidden_channels)

        # 2. HGT Convolution 레이어 스택
        self.convs = nn.ModuleList()
        for _ in range(num_layers):
            conv = HGTConv(
                hidden_channels,
                hidden_channels,
                metadata,
                heads=num_heads
            )
            self.convs.append(conv)

        # 3. 출력 레이어
```

```

self.out_lin=Linear(hidden_channels,out_channels)

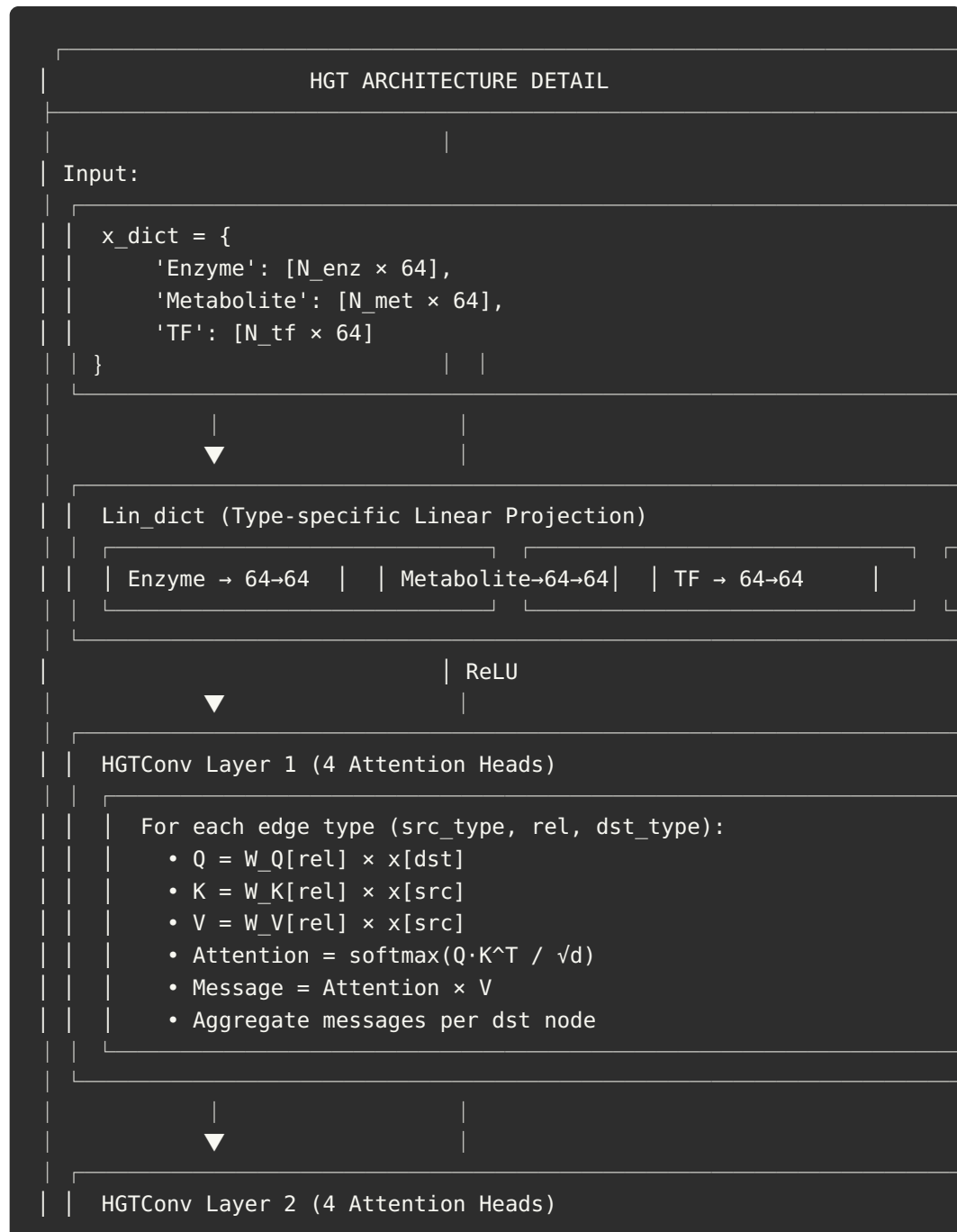
def forward(self,x_dict,edge_index_dict):
    # Step 1: 타입별 특징 투영 + ReLU
    x_dict={
        node_type:self.lin_dict[node_type](x).relu_()
        for node_type,x in x_dict.items()
    }

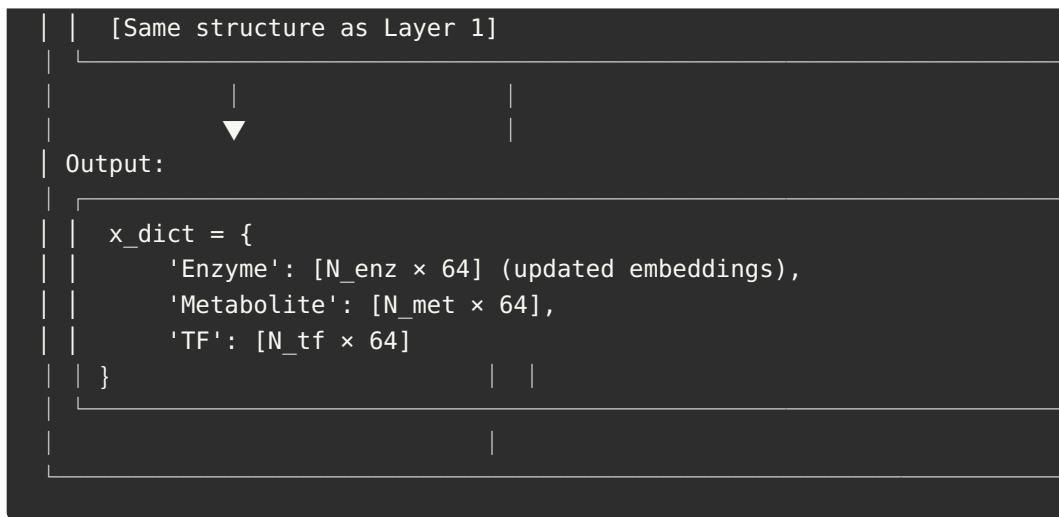
    # Step 2: HGT Convolution 통과
    for conv in self.convs:
        x_dict=conv(x_dict,edge_index_dict)

    return x_dict

```

4.3 HGT 아키텍처 다이어그램





4.4 Link Predictor

```

class LinkPredictor(nn.Module):
    """
    노드 임베딩 쌍으로부터 엣지 존재 확률 예측

    Method: Dot Product Scoring
    score(u, v) = embedding(u) · embedding(v)
    """

    def forward(self, x_src, x_dst, edge_label_index):
        row, col = edge_label_index
        src_feats = x_src[row] # Source node embeddings
        dst_feats = x_dst[col] # Destination node embeddings

        # Element-wise product + sum = dot product
        return (src_feats * dst_feats).sum(dim=-1)
  
```

4.5 하이퍼파라미터 설정

파라미터	값	선택 근거
<code>in_channels</code>	64	노드 특징 차원
<code>hidden_channels</code>	64	모델 용량
<code>out_channels</code>	64	출력 임베딩 차원
<code>num_heads</code>	4	Multi-head attention
<code>num_layers</code>	2	2-hop 이웃 정보 집약
<code>learning_rate</code>	0.01	Adam 옵티마이저

파라미터	값	선택 근거
epochs	10-100	조기 종료

5. 모델 학습 파이프라인

5.1 데이터 분할

파일 위치: `src/trainer.py`

```
# RandomLinkSplit 사용
transform = T.RandomLinkSplit(
    num_val=0.1,  # 10% 검증
    num_test=0.1, # 10% 테스트
    is_undirected=True,
    edge_types=[target_edge_type],
    add_negative_train_samples=False
)

train_data, val_data, test_data = transform(data)
```

5.2 학습 루프

```
def train():
    # 1. 전방향 패스
    x_dict = model(train_data.x_dict, train_data.edge_index_dict)

    # 2. Positive 샘플 점수
    pos_out = predictor(x_dict[src_type], x_dict[dst_type],
                       edge_label_index)

    # 3. Negative 샘플링 (Random)
    neg_dst_idx = torch.randint(0, x_dict[dst_type].size(0),
                                (src_idx.size(0),), device=device)
    neg_out = predictor(x_dict[src_type], x_dict[dst_type],
                       torch.stack([src_idx, neg_dst_idx]))

    # 4. Binary Cross Entropy Loss
    loss = -torch.log(torch.sigmoid(pos_out) + 1e-15).mean() \
           - torch.log(1 - torch.sigmoid(neg_out) + 1e-15).mean()

    # 5. 역전파 및 최적화
    loss.backward()
    optimizer.step()
```

5.3 평가 지표

지표	설명	목표 값
AUC-ROC	Area Under ROC Curve	> 0.85
Hits@K	Top-K 예측 중 실제 양성 비율	> 0.5
MRR	Mean Reciprocal Rank	> 0.3

6. 해석 가능성 분석 (GNNShap)

6.1 목적

GNN 예측의 **블랙박스 문제**를 해결하기 위해, 각 예측에 어떤 엣지가 중요하게 기여했는지 **Shapley Value**를 통해 계산합니다.

6.2 방법론

파일 위치: `src/gnnshap_explainability.py`

```
def compute_shapley_values(data, model, predictor,
                           src_type, src_idx, dst_type, dst_idx,
                           edges, device, n_samples=50):
    """
    Monte Carlo 샘플링으로 Shapley Value 근사 계산

    Algorithm:
    1. K-hop 이웃의 엣지들 수집 (max 20개)
    2. n_samples 번 반복:
        a. 랜덤 순열 생성
        b. 순차적으로 엣지 추가하며 점수 변화 측정
        c. 각 엣지의 marginal contribution 누적
    3. 평균 계산 → Shapley Value
    """

    shapley_values = {i: 0.0 for i in range(n_edges)}

    for _ in range(n_samples):
        perm = np.random.permutation(n_edges)

        prev_score = None
        for i, edge_idx in enumerate(perm):
            # Coalition 구성
            edges_to_mask = [...]
            masked_data = mask_edges(data, edges_to_mask)

            # 점수 계산
```

```

x_dict=model(masked_data.x_dict,masked_data.edge_index_dict)
curr_score=predictor(...).sigmoid().item()

# Marginal contribution
shapley_values[edge_idx] += curr_score - prev_score
prev_score = curr_score

# 평균
for k in shapley_values:
    shapley_values[k] /= n_samples

return shapley_values

```

6.3 Fidelity 검증

```

def compute_explanation_fidelity(..., top_edges):
    """
    설명의 신뢰도 검증

    Fidelity = 원래 점수 - Top-K 엣지 제거 후 점수

    높은 Fidelity = 설명이 실제로 중요한 엣지를 잘 식별함
    """

```

6.4 출력 예시

```

Explaining Enzyme[42] → Metabolite[15] (Score: 0.9234)
Fidelity: 0.4521 (Original: 0.9234 → After: 0.4713)
Top-3 edges:
1. ('Enzyme', 'catalyzes', 'Metabolite') (Shapley: 0.1823)
2. ('TF', 'regulates', 'Enzyme') (Shapley: 0.1156)
3. ('Enzyme', 'interacts', 'Enzyme') (Shapley: 0.0892)

```

7. 분자 도킹 파이프라인

7.1 목적

GNN이 예측한 대사체-효소 결합을 구조적으로 검증합니다.

7.2 도킹 워크플로우

파일 위치: `src/run_docking.py`

Step 1: Structure Retrieval

Receptor: AlphaFold API → PDB
Ligand: KEGG REST API → MOL
(Also: PubChem → SDF)



Step 2: Structure Preparation

OpenBabel:
• PDB → PDBQT (receptor, add H, Gasteiger charges)
• MOL → PDBQT (ligand, 3D generation, add H)



Step 3: Docking Calculation

AutoDock Vina:
• Blind docking (80×80×80 Å grid)
• exhaustiveness = 8
• CPU = 4



Step 4: Results

Output: docked.pdbqt (poses), docking.log (affinities)
Best Affinity: kcal/mol (more negative = stronger binding)

7.3 주요 함수

```
def prepare_receptor(pdb_file, out_pdbqt):
    """PDB → PDBQT 변환 (OpenBabel)"""
    cmd = [OBABEL_PATH, str(pdb_file), "-xr", "-O", str(out_pdbqt),
           "-h", "--partialcharge", "gasteiger"]

def prepare_ligand(mol_file, out_pdbqt):
    """MOL → PDBQT 변환 (3D 구조 생성)"""
    cmd = [OBABEL_PATH, str(mol_file), "-O", str(out_pdbqt),
           "-h", "--gen3d", "--partialcharge", "gasteiger"]

def run_vina(receptor_pdbqt, ligand_pdbqt, out_pdbqt, log_file):
    """AutoDock Vina 실행"""
    cmd = [
        VINA_PATH,
        "--receptor", str(receptor_pdbqt),
        "--ligand", str(ligand_pdbqt),
        "--out", str(out_pdbqt),
        "--center_x", "0", "--center_y", "0", "--center_z", "0",
```

```
--size_x", "80", "--size_y", "80", "--size_z", "80",
"--cpu", "4", "--exhaustiveness", "8"
]
```

7.4 도킹 결과 요약

순 위	리간드	수용체	도킹 점수 (kcal/ mol)	신규성
1	6''-O-Malonyldaidzin	β-Glucosidase (6YN7)	-9.165	★★★★
2	6''-O-Acetyldaidzin	2-HIS (8E83)	-8.863	★★★★
3	6''-O-Acetylgenistin	2-HIS (8E83)	-7.768	★★★★
4	6''-O- Malonylgenistin	2-HIS (8E83)	-7.485	★★★★
5	Daidzin	β-Glucosidase (6YN7)	-8.496	(기존 Km 데이 터)

해석: 도킹 점수 -7 kcal/mol 이하는 생물학적으로 유의미한 결합력으로 간주

8. MD 시뮬레이션 계획

8.1 목적

도킹 결과의 동적 안정성 검증 및 정밀 결합 에너지 계산

8.2 시뮬레이션 프로토콜

파일 위치: md_protocol/mdp/

```
MD SIMULATION PROTOCOL

Phase 1: System Preparation
├── 리간드 파라미터화: GAFF2 / CGenFF
├── 시스템 구축: protein + ligand + water + ions
└── 토폴로지 생성: GROMACS pdb2gmx / AMBER tleap
```

```
| Phase 2: Energy Minimization (em.mdp)
| |─ Algorithm: Steepest Descent
| |─ Steps: 50,000
| |─ Tolerance: 1000 kJ/mol/nm
|
| Phase 3: NVT Equilibration (nvt.mdp)
| |─ Duration: 100 ps
| |─ Temperature: 300 K
| |─ Thermostat: V-rescale
| |─ Restraints: Protein backbone
|
| Phase 4: NPT Equilibration (npt.mdp)
| |─ Duration: 100 ps
| |─ Pressure: 1 bar
| |─ Barostat: Parrinello-Rahman
| |─ Restraints: Protein backbone (reduced)
|
| Phase 5: Production MD (md_100ns.mdp)
| |─ Duration: 100 ns
| |─ Time step: 2 fs
| |─ Output: 10 ps intervals
| |─ Constraints: LINCS (H-bonds)
|
| Phase 6: Analysis
| |─ RMSD: Ligand stability
| |─ RMSF: Binding site flexibility
| |─ H-bonds: Protein-ligand hydrogen bonds
| |─ MM-PBSA/GBSA: Binding free energy
|
```

8.3 예상 산출물

산출물	형식	설명
RMSD plot	PNG/PDF	리간드 RMSD 시간 변화 (수렴 확인)
Binding ΔG	표	MM-PBSA 결합 자유 에너지 $\pm$ 오차
Key residues	목록	상호작용 기여도 상위 10개 잔기
H-bond analysis	그래프	수소결합 개수 시간 변화
Representative snapshot	PDB	안정 상태 대표 구조

8.4 예상 일정

단계	시스템 3개 기준 소요 시간
시스템 준비	1일

단계	시스템 3개 기준 소요 시간
Equilibration	0.5일
Production MD	3-5일 (GPU)
분석	1-2일
총계	5-8일

9. 검증 및 평가 지표

9.1 GNN 모델 평가

지표	공식	목표
AUC-ROC	Area under ROC curve	≥ 0.85
Hits@10	True positives in top 10 / Total positives	≥ 0.5
MRR	Mean(1/rank of first true positive)	≥ 0.3
Fidelity	Score drop when removing important edges	> 0.3

9.2 경로 분석 통계

분석	방법	기준
Pathway Enrichment	Fisher's Exact Test	$P < 0.05$
Multiple Testing	Benjamini-Hochberg FDR	$FDR < 0.1$
Effect Size	Odds Ratio + 95% CI	$OR > 2$

9.3 도킹/MD 품질 지표

지표	기준	해석
Docking Score	< -7 kcal/mol	유의미한 결합
Ligand RMSD	$< 3 \text{ \AA}$ (수렴)	안정적 결합
MM-PBSA ΔG	< -5 kcal/mol	강한 결합

지표	기준	해석
H-bond occupancy	> 30%	안정적 상호작용

10. 파일 구조 및 재현성

10.1 프로젝트 구조

```

/data/ethylene/
├── data/
│   ├── processed/          # 전처리된 데이터
│   │   ├── mtbls531_differential.csv
│   │   ├── pxd006989_differential.csv
│   │   ├── graph.pt        # PPI 그래프
│   │   ├── bipartite_graph.pt # 이중 그래프
│   │   └── strict_bipartite_v2.pt
│   └── structures/         # 구조 파일
│       ├── pdb/            # 수용체 PDB
│       └── ligands/         # 리간드 SDF
├── src/                    # 핵심 스크립트 (78개)
│   ├── model.py            # GNN 모델 정의
│   ├── trainer.py          # 학습 루프
│   ├── bipartite_builder.py # 그래프 구축
│   ├── run_docking.py      # 도킹 파이프라인
│   ├── gnnshap_explainability.py # 해석 가능성
│   └── ...
├── md_protocol/            # MD 시뮬레이션
│   └── mdp/                # MDP 설정 파일
│       ├── em.mdp
│       ├── nvt.mdp
│       ├── npt.mdp
│       └── md_100ns.mdp
├── results/                # 결과물
│   ├── docking/            # 도킹 결과
│   ├── figures/            # 생성된 그림
│   └── explainability/     # GNNShap 결과
└── docs/                   # 문서
    └── MANUSCRIPT_*.md     # 논문 초안

```

10.2 재현 스크립트

```

# 1. 환경 설정
conda create -n ethylene python=3.10
conda activate ethylene
pip install -r requirements.txt

```

```
# 2. 그래프 구축
python src/bipartite_builder.py \
  --graph data/processed/graph.pt \
  --output data/processed/bipartite_graph.pt

# 3. GNN 학습
python src/trainer.py

# 4. 해석 가능성 분석
python src/gnnshap_explainability.py

# 5. 분자 도킹
python src/run_docking.py \
  --input results/gnn/top_candidates.csv \
  --outdir results/docking

# 6. 그림 생성
python src/generate_pathway_figures.py
python src/generate_enhanced_figures.py
```

10.3 의존성

```
# requirements.txt
torch>=2.0
torch-geometric>=2.4
pandas>=2.0
numpy>=1.24
matplotlib>=3.7
scipy>=1.10
scikit-learn>=1.2
requests>=2.28
tqdm>=4.65
```

☒ 논의 포인트

연구 방법론 관련

1. **GNN 모델 선택**: HGT vs 다른 아키텍처 (GAT, HAN)?
2. **특징 벡터 초기화**: Random vs 분자 기술자 (Morgan fingerprint, ESM-2)?
3. **Negative Sampling 전략**: Random vs Hard negative mining?

검증 관련

1. **도킹 결과 해석**: 점수 cutoff -7 vs -8 kcal/mol?
2. **MD 시뮬레이션 시간**: 100 ns 충분? 200 ns 필요?

3. 실험적 검증: 효소 활성 측정, SPR/ITC 결합 측정?

확장 관련

1. 다른 호르몬 처리: JA (자스몬산), SA (살리실산) 추가?
2. 다른 조직: 콩잎 외에 뿌리, 종자 데이터?

참고 문헌

1. Yuk et al. (2016) "Ethylene Induced High Accumulation of Dietary Isoflavones..." J. Agric. Food Chem.
2. Hu et al. (2020) "Heterogeneous Graph Transformer" WWW 2020
3. Trott & Olson (2010) "AutoDock Vina: Improving the speed and accuracy of docking" J. Comput. Chem.
4. Lundberg & Lee (2017) "A Unified Approach to Interpreting Model Predictions" NeurIPS

작성자: Claude AI

검토 요청: 동료 연구자

다음 단계: MD 시뮬레이션 수행 후 결과 업데이트